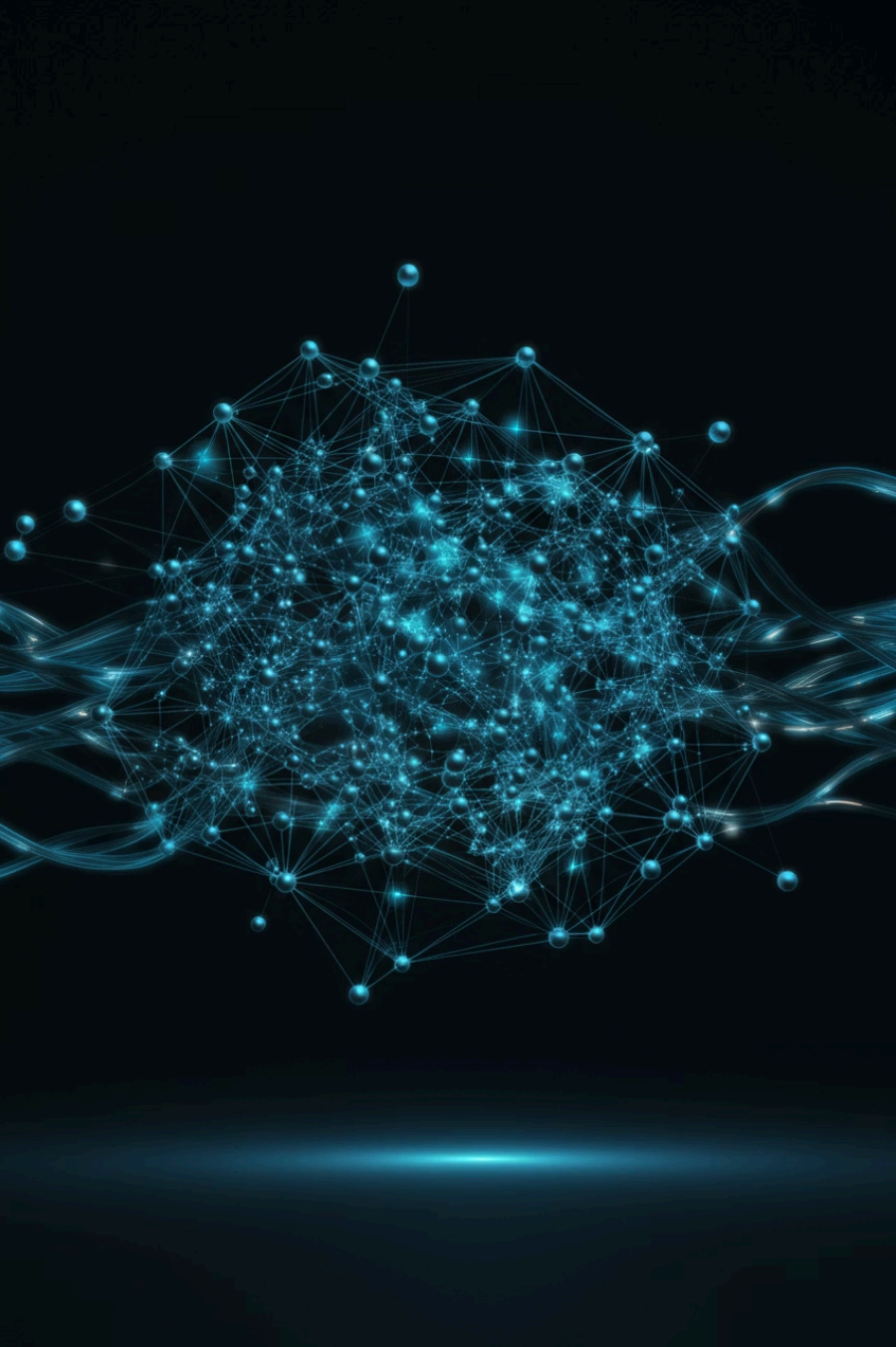




NLP Fundamentals

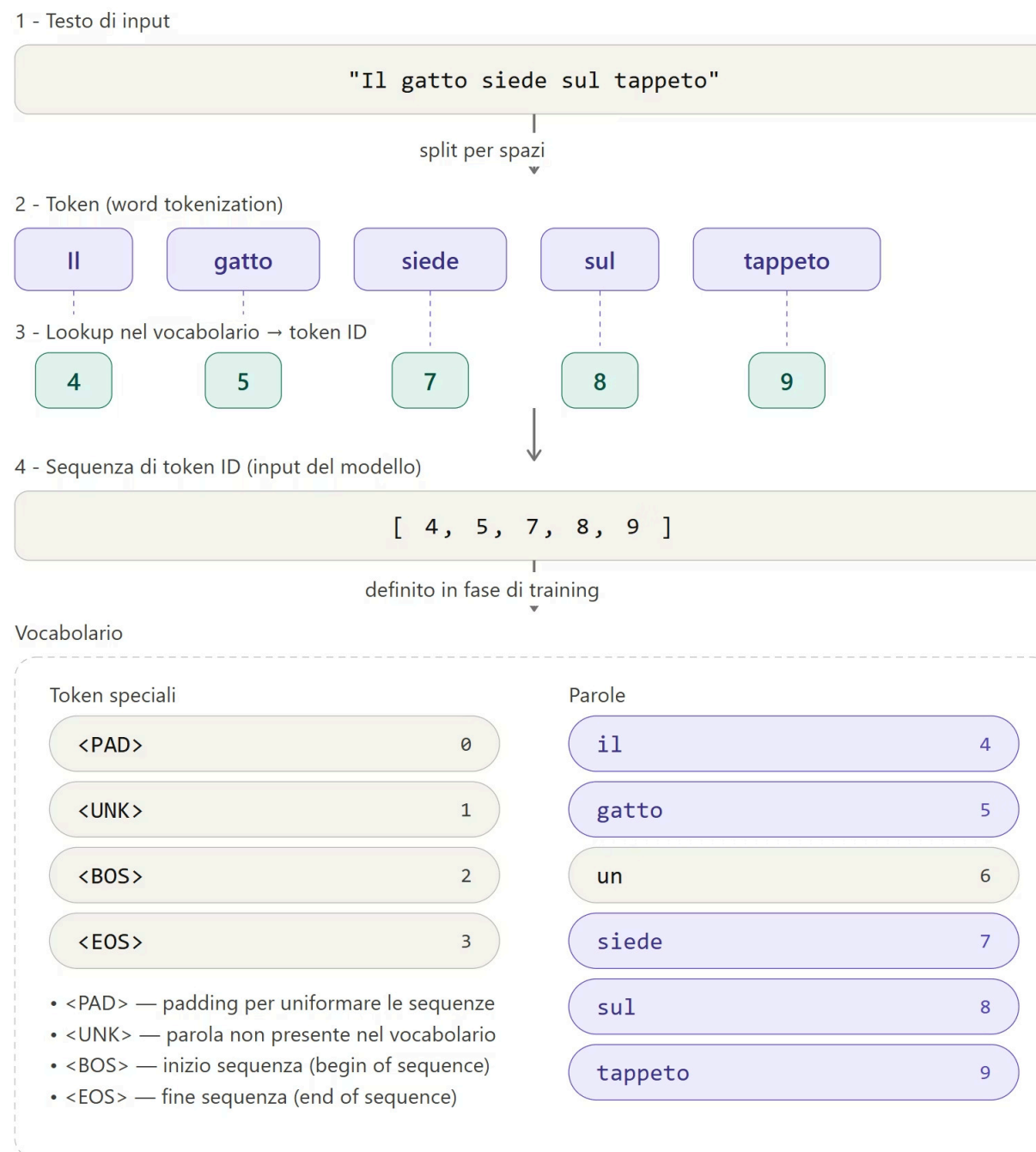
Dal testo grezzo alla rappresentazione semantica.



by Daniele Mantovani

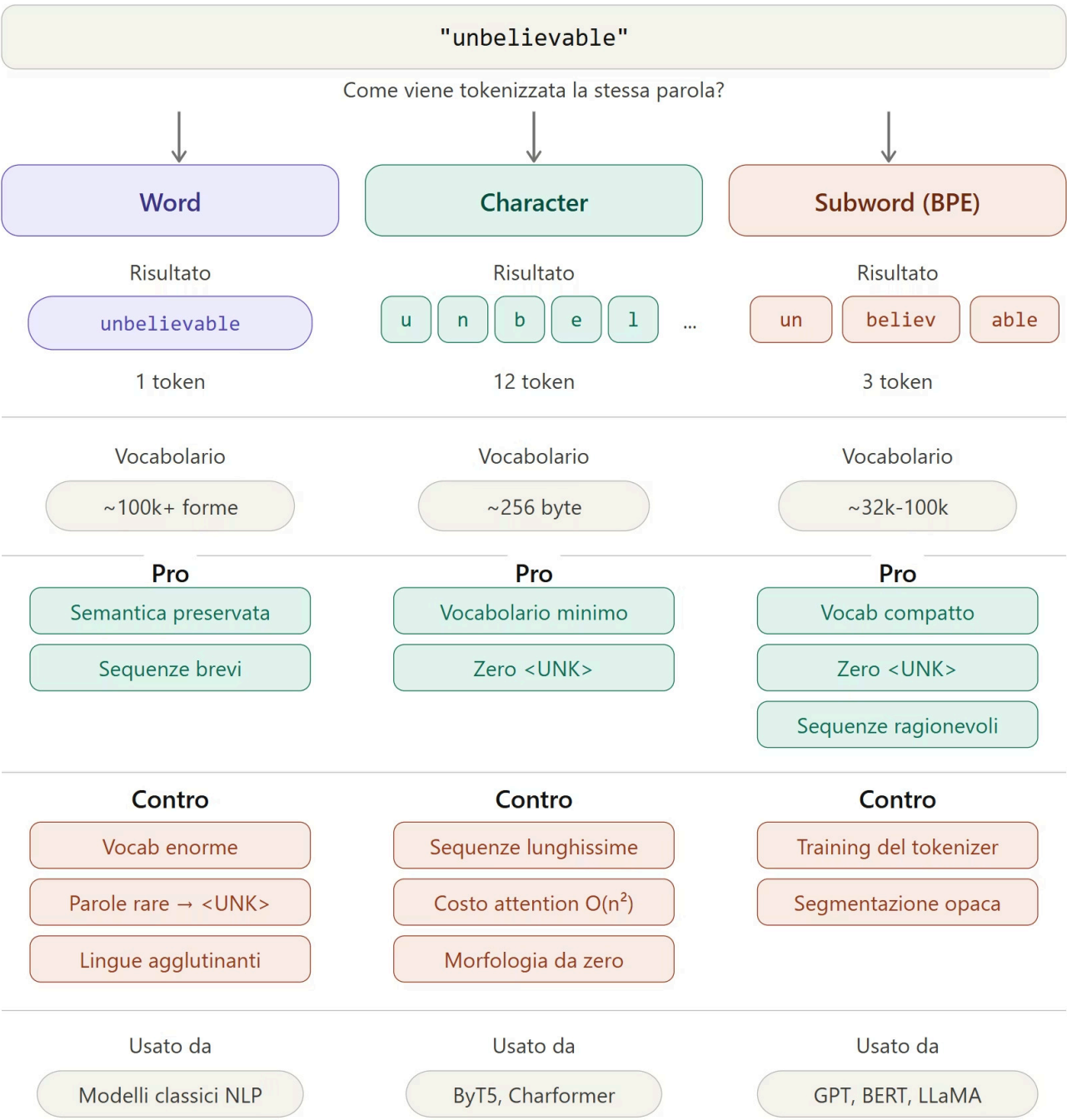
Dal testo ai numeri: come leggono i LLM

- Un Large Language Model non vede lettere o parole: vede solo sequenze di numeri interi.
- La tokenizzazione è il processo che trasforma il testo grezzo — qualsiasi testo, in qualsiasi lingua — in una sequenza di token ID, ciascuno corrispondente a una voce in un vocabolario finito.
- È il primo anello della pipeline e ogni scelta fatta qui (granularità, dimensione del vocabolario, algoritmo) si propaga fino all'output del modello.



Strategie di Tokenizing

- Esistono tre strategie fondamentali per segmentare il testo in token.
- Nessuna è "la migliore" in assoluto: ciascuna rappresenta un punto diverso nel trade-off tra dimensione del vocabolario, lunghezza delle sequenze e capacità di gestire parole mai viste.



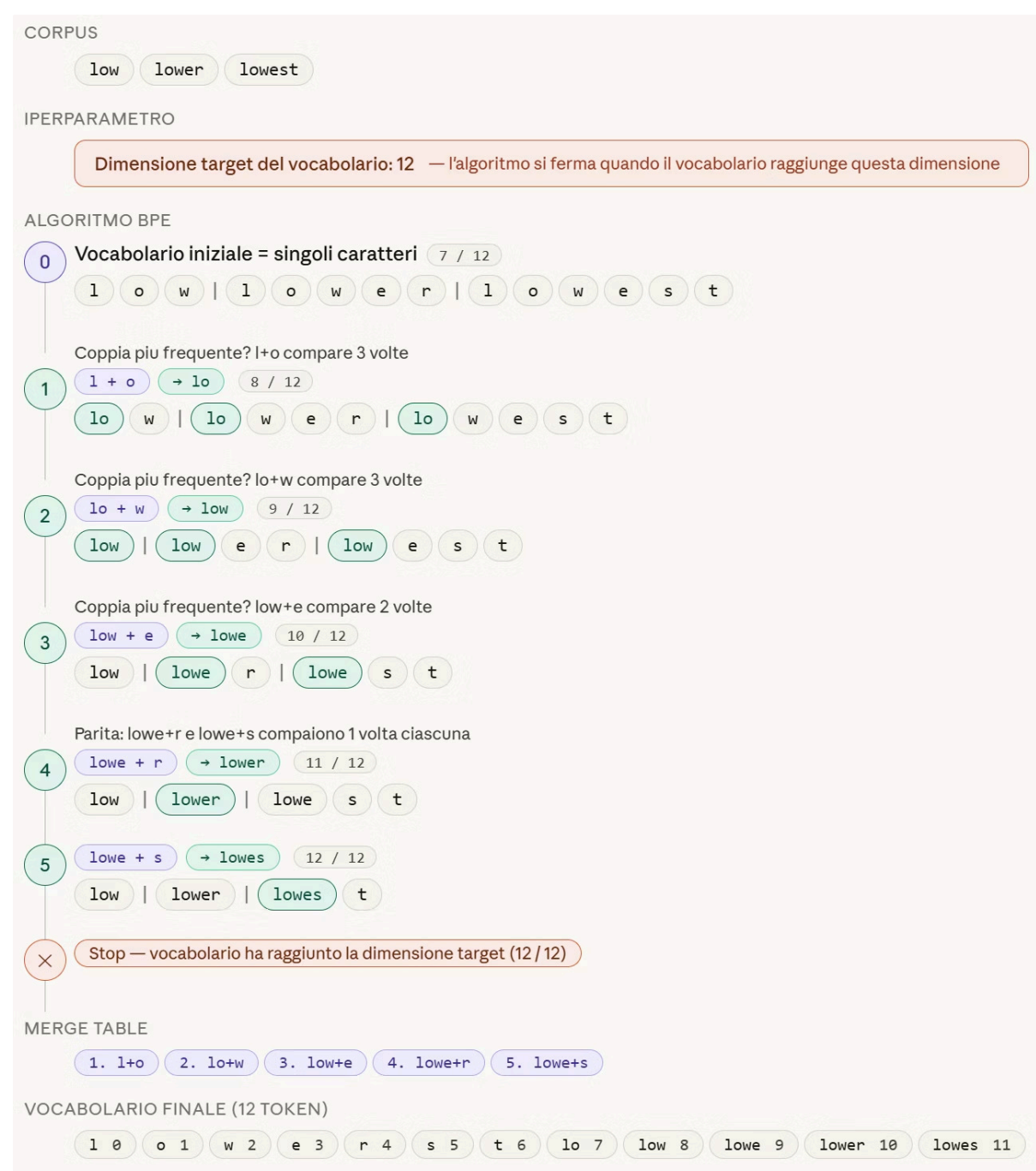
[OpenAI Tokenizer](#)

[TikTokenizer](#)

[tiktoken](#)

BPE – Byte Pair Encoding

- Byte Pair Encoding e sue varianti sono tra i metodi più diffusi di subword tokenization:
 - GPT models → usano varianti come **byte-level BPE / tiktoken**
 - LLaMA → usa **SentencePiece (BPE-like)**
 - Mistral → usa anch'esso **SentencePiece / Unigram**
- L'idea è semplice: partire dai singoli caratteri e fondere iterativamente la coppia più frequente, fino a raggiungere la dimensione desiderata del vocabolario o fino a quando non ci sono più merge.
- La condizione di stop di BPE è **il raggiungimento della dimensione target del vocabolario**, che viene fissata come iperparametro prima del training. Ad esempio, se decidiamo che il vocabolario deve avere 10.000 token, l'algoritmo parte dai ~256 byte iniziali e continua a fondere coppie fino a quando il vocabolario non contiene esattamente 10.000 voci. A quel punto si ferma, indipendentemente da quante coppie frequenti restano ancora da fondere.



- ❏ Il nome "Byte Pair Encoding" deriva dall'algoritmo originale (Gage, 1994), che operava su byte grezzi. L'adattamento per NLP (Sennrich et al., 2016) applica la stessa idea partendo da caratteri Unicode. Nella pratica, modelli come GPT-2 e GPT-4 usano una variante chiamata byte-level BPE, che applica BPE a una rappresentazione a livello di byte del testo, permettendo di gestire qualsiasi input senza preprocessing.

Perché i token ID non bastano come input del modello?

Dopo la tokenizzazione abbiamo una sequenza di numeri interi — ad esempio [4, 5, 7, 8, 9]. Sembra già un input numerico pronto per una rete neurale. Ma usare questi ID direttamente sarebbe un errore concettuale grave. Vediamo perché.

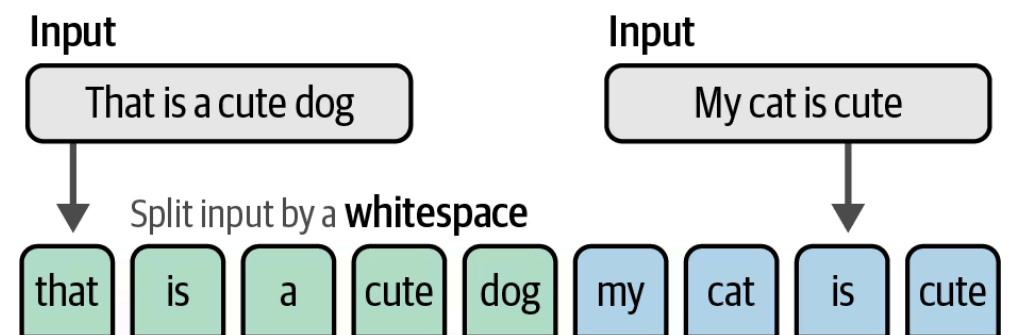
- Gli ID sono **arbitrari**: "gatto" = 5 e "tappeto" = 9 non significa che "tappeto" sia "di più" o "più lontano" di "gatto"
- Le operazioni matematiche sugli ID sono **prive di senso**: la media tra "gatto" (5) e "siede" (7) non è "un" (6)
- Un modello che riceve ID grezzi **imporrebbe relazioni d'ordine e distanza** tra parole che non esistono

Il problema di fondo è che i token ID sono **etichette categoriali** mascherate da numeri: il fatto che siano interi è solo una convenienza di rappresentazione, non porta informazione semantica.

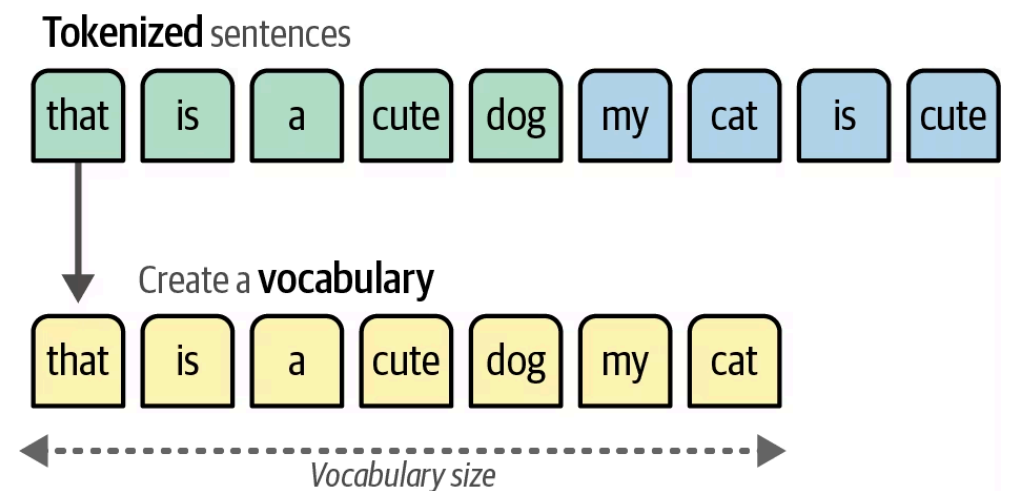
One-Hot Encoding / Bag-of-Words

- **La Logica:** Per trattare i token come entità distinte e non come numeri ordinati, trasformiamo ogni ID in un vettore indipendente.
- **Definizione:** Creiamo un vettore di dimensione $|V|$ (grandezza del vocabolario, es. 10.000) pieno di zeri.
- **L'Operazione:** Poniamo un "1" esclusivamente nella posizione corrispondente all'ID del token.
- **Risultato (Ortogonalità):** Ogni parola ha il suo asse. Il prodotto scalare tra due parole diverse è sempre 0.
- *Matematica:* $Token_A \cdot Token_B = 0$ (se $A \neq B$).
- *Significato:* La macchina sa che sono diversi, ma non sa *quanto* sono diversi (**nessuna misura della similarità**).

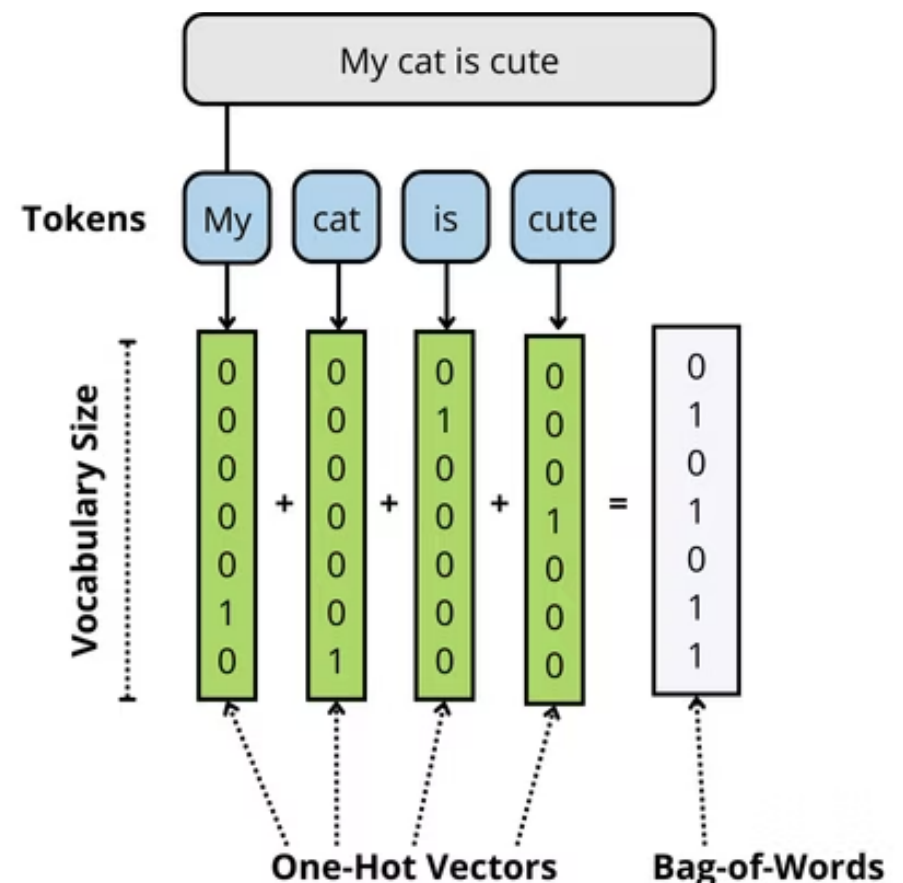
1. Tokenizing



2. Creazione del vocabolario



3. Costruzione della Bag-of-Words



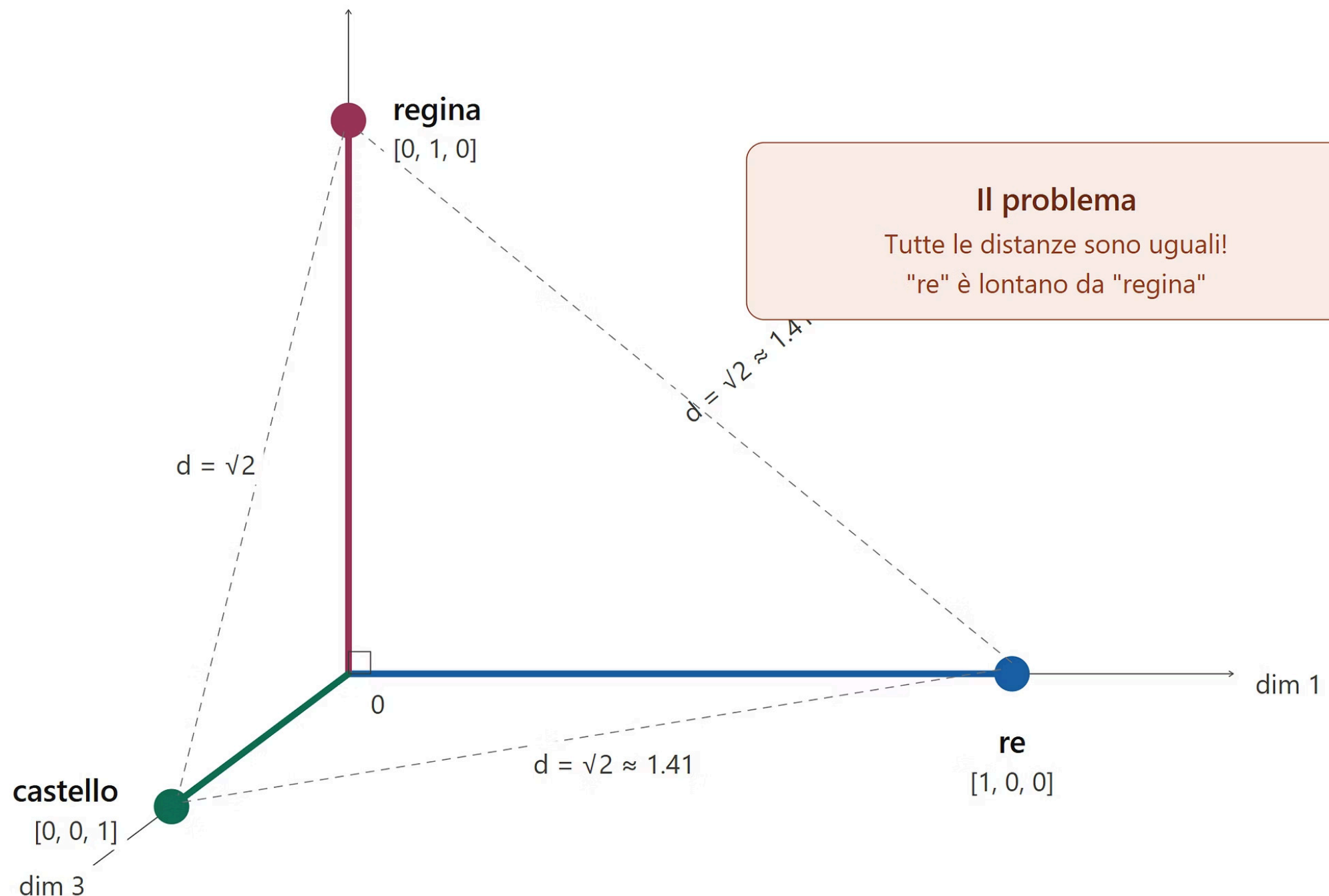
One-hot encoding non sa nulla di significato

Ricordiamo: in one-hot encoding ogni parola è un vettore con un solo 1 e tutti 0.

- Se $|V| = 50,000$, ogni parola è un vettore di 50.000 dimensioni
- $\text{re} = [1, 0, 0, 0, \dots]$, $\text{regina} = [0, 1, 0, 0, \dots]$, $\text{castello} = [0, 0, 1, 0, \dots]$
- La **distanza** tra "re" e "regina" è **identica** a quella tra "re" e "castello"
- Tutti i vettori sono **ortogonali**: $\cos(\vec{v}_i, \vec{v}_j) = 0 \quad \forall i \neq j$

Spazio one-hot (3 parole → 3 dimensioni)

Ogni parola vive su un asse separato — nessuna relazione catturata

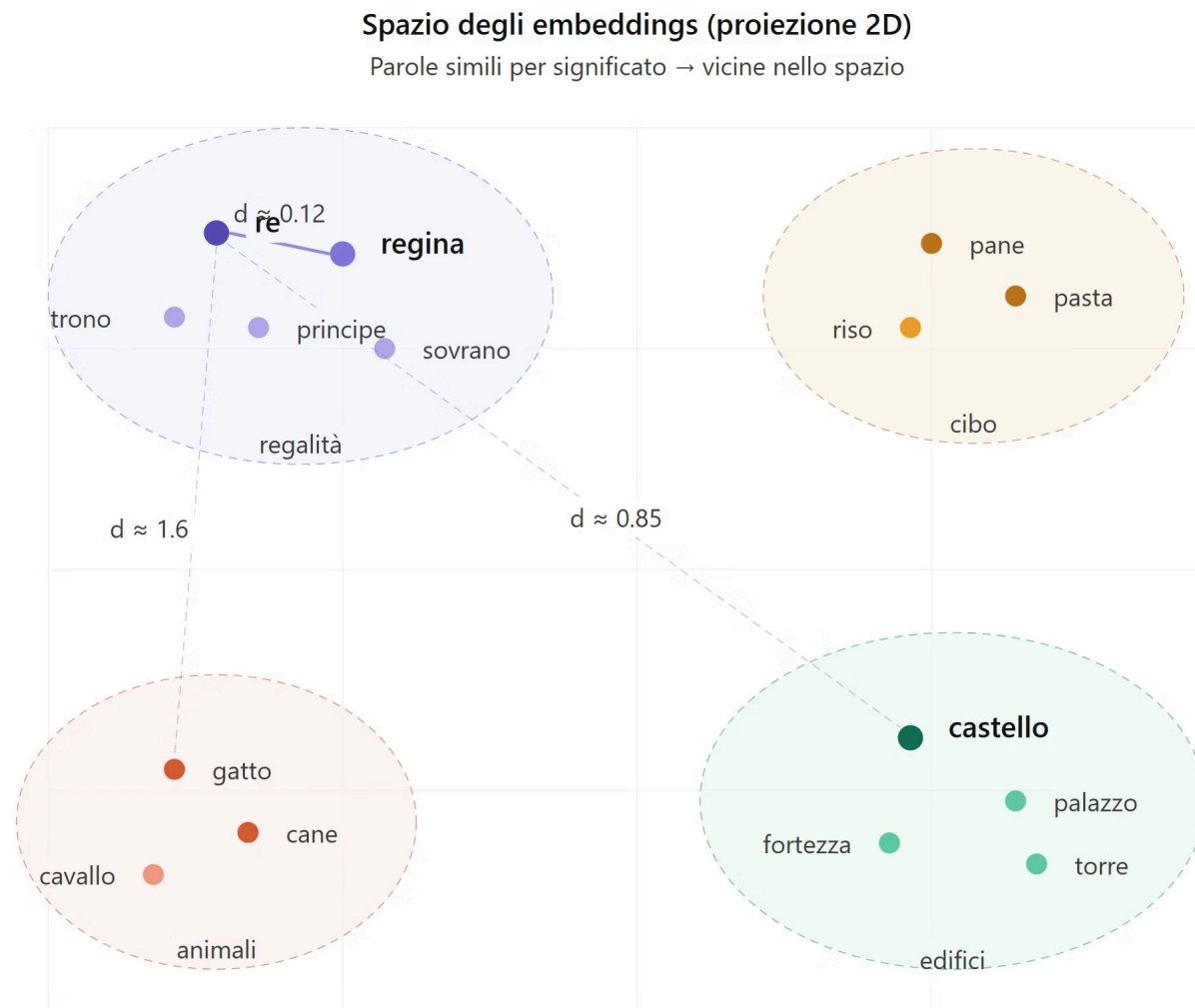


L'intuizione: e se la posizione nello spazio avesse significato?

Immaginiamo un mondo diverso: uno spazio a **poche dimensioni** (es. 300 anziché 50.000) dove ogni parola è un punto, e **la distanza tra punti riflette la similarità di significato**.

- Parole con significato simile → vicine nello spazio ("re" vicino a "regina", "principe", "sovrano")
- Parole con significato diverso → lontane ("re" lontano da "gatto", "pasta", "automobile")
- Le **direzioni** nello spazio codificano relazioni (la direzione da "uomo" a "donna" è simile alla direzione da "re" a "regina")

Questo è esattamente ciò che fanno gli **embeddings**: mappano ogni parola in un vettore denso in $\mathbb{R}^d$ (tipicamente d tra $\sim 2k$ e $\sim 8k$) dove la geometria cattura la semantica.



Distanza nello spazio $\approx$ differenza di significato. Più vicini = più simili.

Formalizzazione: cos'è un Embedding

Un **Word Embedding** è una funzione:

$$E : V \rightarrow \mathbb{R}^d$$

che mappa ogni parola w del vocabolario V in un vettore denso $\vec{e}_w$ di dimensione d .

Proprietà desiderate:

- **Densità**: ogni componente del vettore ha un valore reale continuo (non più 0/1)
- **Bassa dimensionalità**: $d \ll |V|$ — tipicamente $d \in [2k, 8k]$
- **Similarità geometrica**: $\text{sim}(w_1, w_2) \approx \cos(\vec{e}_{w_1}, \vec{e}_{w_2})$

Esempio concreto:

$$\vec{e}_{\text{re}} = [0.21, -0.45, 0.83, \dots, 0.12] \in \mathbb{R}^{300}$$

❏ Il vettore non è progettato a mano — viene **appreso dai dati**. Le singole dimensioni **non hanno** un significato interpretabile diretto (non c'è una dimensione per "regalità" e una per "genere"), ma le relazioni geometriche emergono dal training.

❏ Abbiamo parlato di Word Embedding, ma lo stesso concetto si applica ai Token (Embedding).

La distributional hypothesis

Ma come si decide dove posizionare le parole nello spazio? L'idea fondamentale è la **distributional hypothesis** (Harris, 1954; Firth, 1957):

"You shall know a word by the company it keeps." — J.R. Firth

In altre parole: **parole che appaiono in contesti simili hanno significati simili.**

- "Il **re** governava il regno" / "La **regina** governava il regno" → "re" e "regina" condividono molti contesti
- "Il **gatto** dormiva sul divano" → "gatto" appare in contesti molto diversi da "re"

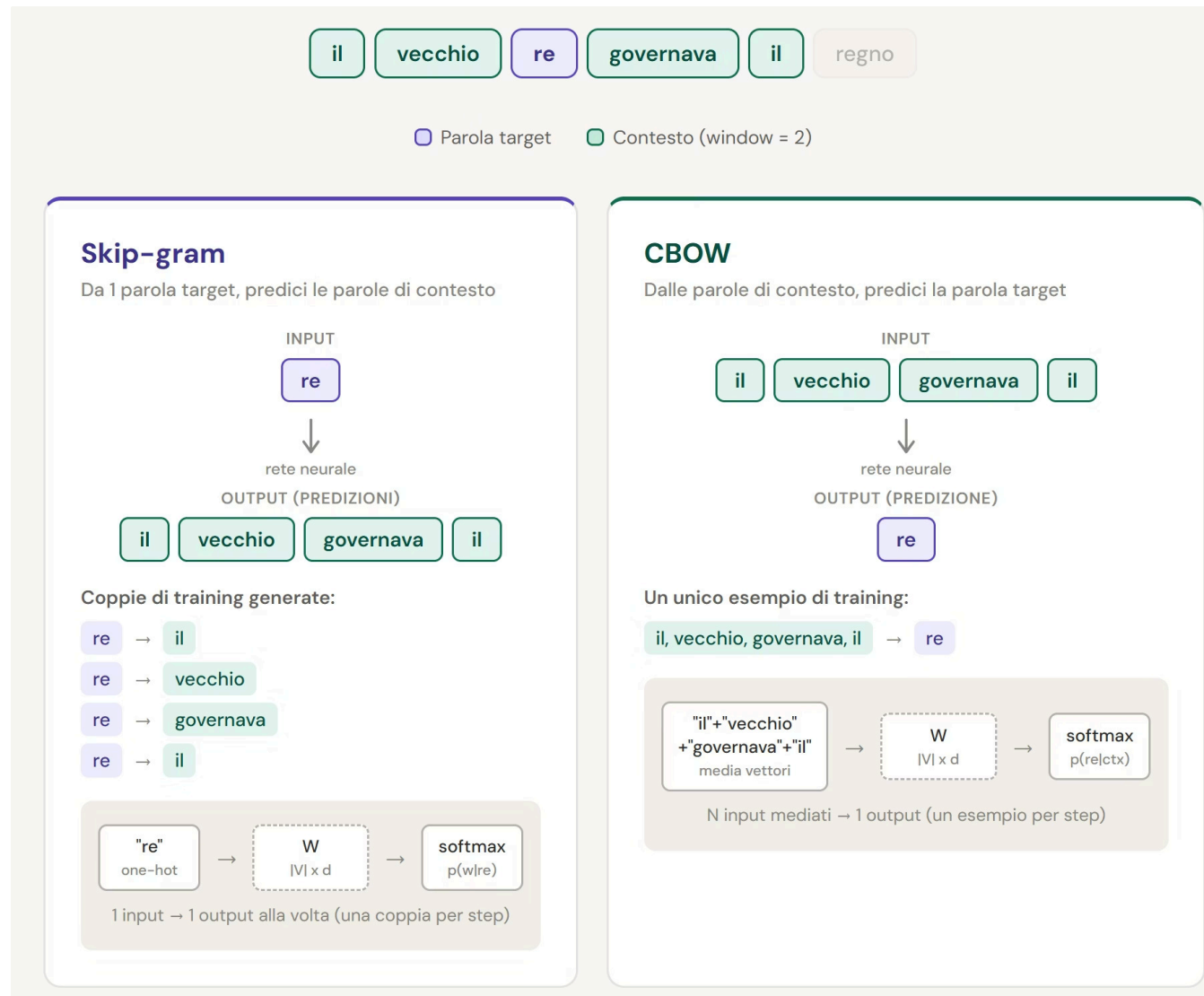
Questa ipotesi è il fondamento teorico di tutti gli algoritmi di apprendimento degli embeddings: si impara la rappresentazione di una parola **osservando i contesti in cui appare e predicando parole vicine** in grandi corpora di testo.

❏ Questa è un'ipotesi — funziona molto bene in pratica ma ha limiti (es. antonimi come "caldo" e "freddo" appaiono negli stessi contesti ma hanno significato opposto).

Word2Vec: come si imparano gli embeddings

Word2Vec (Mikolov et al., 2013) è stato l'algoritmo che ha reso gli embeddings mainstream nel NLP.

Due architetture:

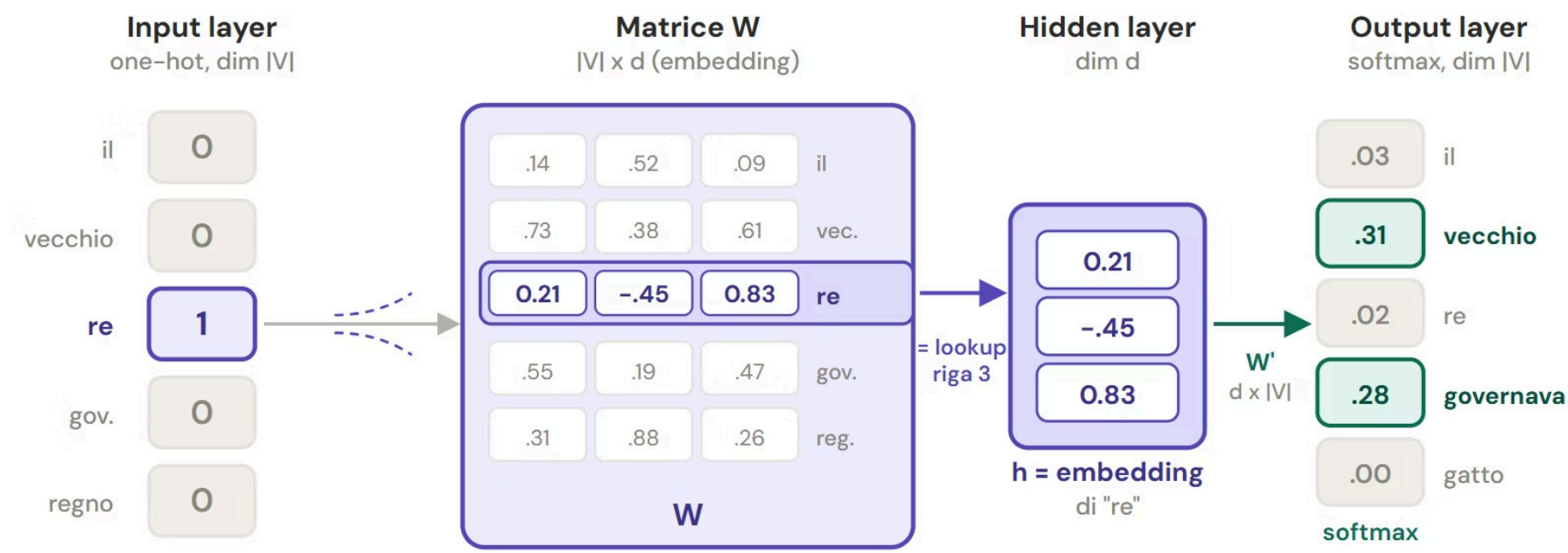


Architettura della rete Word2Vec

Una rete a un solo hidden layer il cui scopo non è la predizione, ma i pesi appresi.

Forward pass (Skip-gram)

Esempio: input "re" con vocabolario $|V| = 5$, embedding dim $d = 3$



I 4 passi del forward pass

1

Codifica one-hot

La parola "re" (indice 3 nel vocabolario) diventa un vettore con tutti zeri tranne un 1 in posizione 3. Il vettore ha dimensione $|V|$ (qui 5).

$$x_{\text{re}} = [0, 0, 1, 0, 0]$$

2

Lookup in W (= embedding)

Moltiplicare il one-hot per W seleziona la riga 3. Il risultato è un vettore denso di dimensione d : questo è l'embedding di "re".

$$h = x \cdot W = W[3, :] = [0.21, -0.45, 0.83]$$

3

Proiezione con W'

L'embedding viene moltiplicato per una seconda matrice W' ($d \times |V|$) per ottenere uno score per ogni parola del vocabolario.

$$s = h \cdot W' \in \mathbb{R}^{|V|}$$

4

Softmax e loss

Gli score diventano probabilità con la softmax. La loss penalizza la rete se assegna bassa probabilità alle vere parole di contesto.

$$p(w|\text{re}) = \frac{e^{s_w}}{\sum_j e^{s_j}}$$

Dopo il training: cosa teniamo?

W si tiene

Ogni riga della matrice W è l'embedding della parola corrispondente. Parole usate in contesti simili ("re" e "regina") convergono verso righe simili nello spazio vettoriale.

W' si scarta

La matrice di output ha servito solo per il training: produrre la distribuzione softmax. Non viene usata come rappresentazione delle parole.

La rete non serve per predire

Word2Vec usa la predizione del contesto come pretesto per il training. Il vero obiettivo non è la predizione corretta, ma i pesi W appresi durante l'ottimizzazione.

La finestra di contesto

Un parametro fondamentale è la **window (o context) size** — quante parole attorno al target consideriamo come contesto.

Esempio con window size = 2:

"**Il vecchio** *re* **governava il** regno con saggezza"

Posizione	1	2	3	4	5	6	7	8
Parola	Il	vecchio	<i>re</i>	governava	il	regno	con	saggezza

Per la parola target "*re*", il contesto è: {il, vecchio, governava, il} (le parole entro 2 posizioni).

- **Finestra piccola** (2-5): cattura relazioni **sintattiche** (aggettivo-nome, verbo-oggetto)
- **Finestra grande** (5-15): cattura relazioni **tematiche/semantiche** (topic, dominio)

Proprietà emergenti

Il risultato più celebre e sorprendente degli embeddings è che le **relazioni semantiche diventano operazioni vettoriali**:

$$\vec{e}_{\text{re}} - \vec{e}_{\text{uomo}} + \vec{e}_{\text{donna}} \approx \vec{e}_{\text{regina}}$$

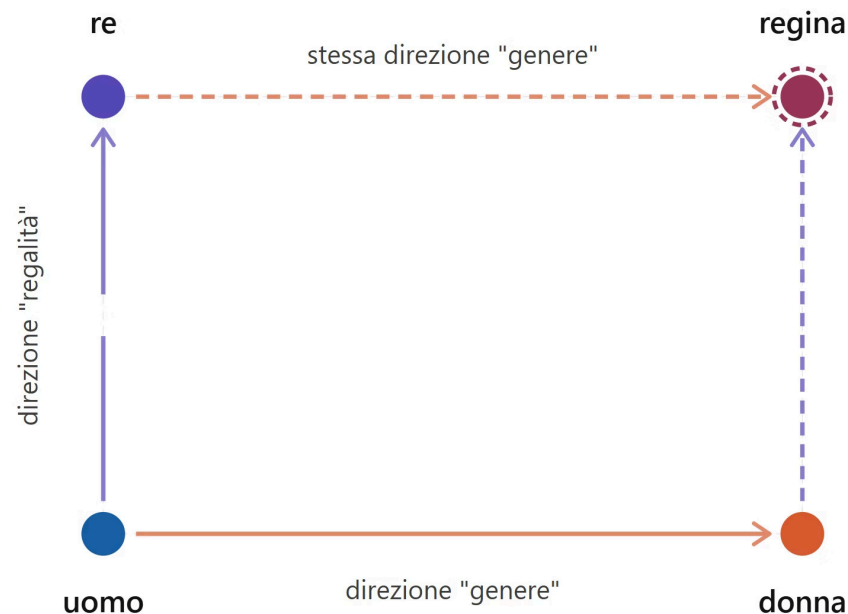
Questo funziona perché la direzione $\vec{e}_{\text{re}} - \vec{e}_{\text{uomo}}$ codifica il concetto di "regalità", e sommandola a $\vec{e}_{\text{donna}}$ si ottiene "regalità + femminile" $\approx$ "regina".

Altri esempi classici:

- $\vec{e}_{\text{Parigi}} - \vec{e}_{\text{Francia}} + \vec{e}_{\text{Italia}} \approx \vec{e}_{\text{Roma}}$ (capitale-nazione)
- $\vec{e}_{\text{walking}} - \vec{e}_{\text{walk}} + \vec{e}_{\text{swim}} \approx \vec{e}_{\text{swimming}}$ (coniugazione)

Aritmetica dei vettori: re - uomo + donna $\approx$ regina

Le relazioni semantiche diventano direzioni nello spazio



$$v(\text{re}) - v(\text{uomo}) + v(\text{donna}) \approx v(\text{regina})$$

Misurare la similarità: cosine similarity

Per confrontare due embedding usiamo la **cosine similarity**:

$$\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| \cdot ||\vec{b}||}$$

- $\cos = 1$: vettori paralleli (stessa direzione → significato identico)
- $\cos = 0$: vettori ortogonali (nessuna relazione)
- $\cos = -1$: vettori opposti (raro in pratica)

Perché cosine e non distanza euclidea? Perché la cosine similarity è **invariante rispetto alla norma del vettore** — conta solo la direzione, non la lunghezza. Parole frequenti tendono ad avere vettori con norma maggiore, ma non necessariamente più significato.

Da word embedding a token embedding

Finora abbiamo parlato di **word embedding**: un vettore per ogni parola intera. Ma nella lezione precedente abbiamo visto che i tokenizer moderni (BPE, WordPiece) spezzano le parole in **subword token**:

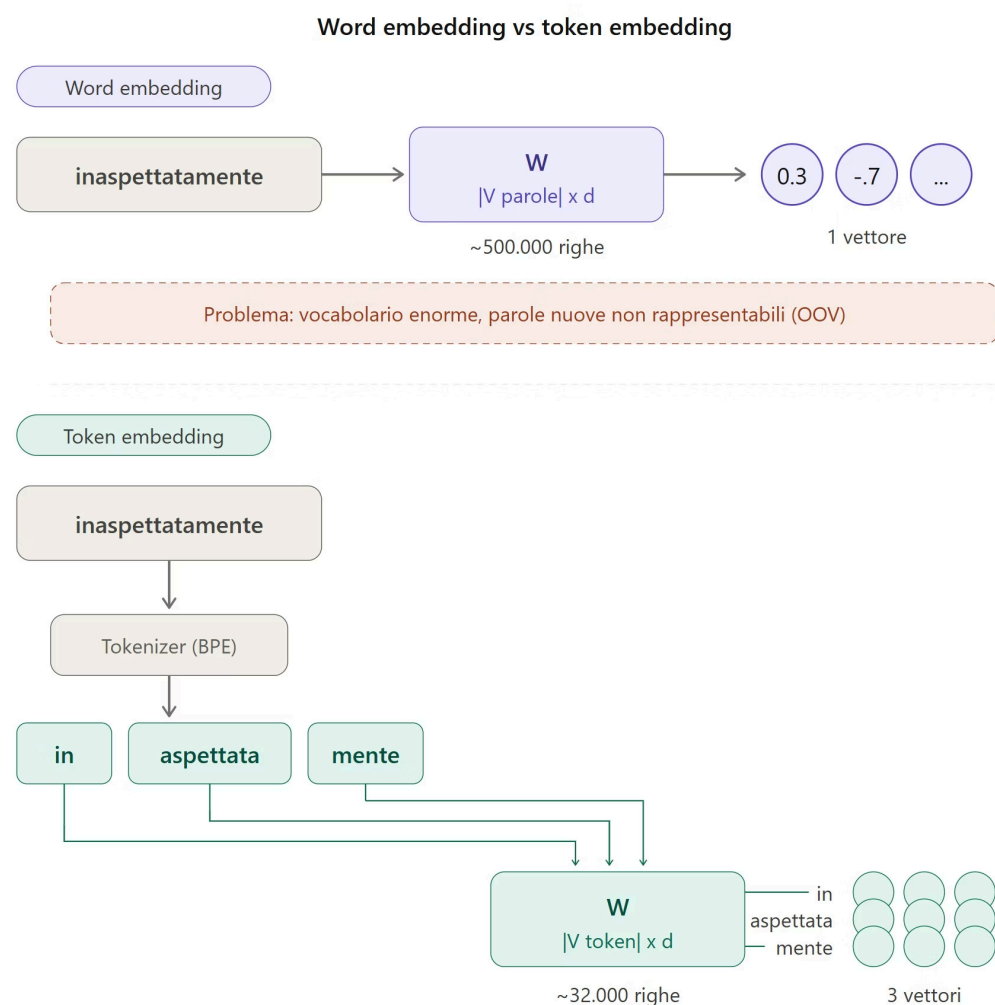
- "inaspettatamente" → ["in", "##aspettata", "##mente"]
- "unhappiness" → ["un", "happiness"] oppure ["un", "happ", "iness"]

Il principio è identico: invece di una matrice $W \in \mathbb{R}^{|V_{\text{parole}}| \times d}$ abbiamo una matrice $W \in \mathbb{R}^{|V_{\text{token}}| \times d}$ — la **token embedding table**.

Vantaggi del passaggio da parole a token:

- **Vocabolario gestibile**: ~30.000-50.000 token vs centinaia di migliaia di parole
- **Nessuna parola "sconosciuta"**: qualsiasi parola nuova si decompone in subword già presenti nella tabella
- **Condivisione morfologica**: i token "in", "##aspettata", "##mente" portano informazione composizionale

Nei transformer (BERT, GPT), il primo layer è proprio una token embedding table: ogni token ID viene mappato alla riga corrispondente — esattamente come il lookup di Word2Vec.



Oltre le parole

L'idea degli embeddings si estende ben oltre le singole parole:

- **Sentence embeddings:** un intero enunciato $\rightarrow$ un vettore in R^d (modelli come Sentence-BERT, modelli di embedding di OpenAI e Anthropic)
- **Document embeddings:** un documento intero $\rightarrow$ un vettore
- **Image embeddings:** un'immagine $\rightarrow$ un vettore (CLIP, modelli vision)
- **Multimodal embeddings:** testo e immagini nello stesso spazio (CLIP mappa "foto di un gatto" e l'immagine di un gatto vicini)

Applicazione fondamentale — **Retrieval-Augmented Generation (RAG):**

1. Si calcola l'embedding di ogni documento in un database
2. Data una query, si calcola il suo embedding
3. Si trovano i documenti con embedding più simile (nearest neighbor search)
4. Si passa il contesto recuperato a un LLM per generare la risposta

[Gemini Embedding 2](#)

Limiti di Word2Vec: una parola, un vettore

Word2Vec assegna un **unico vettore** a ogni parola, indipendentemente dal contesto. Questo crea problemi con:

- **Polisemia**: "pesca" (il frutto) vs "pesca" (l'attività di pescare) → stesso vettore!
- **Composizionalità**: il significato di "non male" non è la somma di "non" + "male"

Questi limiti vengono superati dai **contextual embeddings** (ELMo, BERT, GPT), dove la rappresentazione di una parola dipende dall'intera frase in cui appare.

Riferimenti

[Hands-On Large Language Models](#)

[Chapter 2 - Tokens and Token Embeddings](#)

Word Embeddings

[Word Embeddings](#)

[Word2Vec Training](#)

[tiktoken](#)

Audio Embeddings

[Audio Embeddings](#)

[Audio d'esempio](#)

Image Embeddings

[Image Embeddings](#)

[Immagini d'esempio](#)